PRACTICAL – 12

Travelling Salesman Problem (TSP)

Objective

To write a C++ program to solve the Travelling Salesman Problem using the brute-force method and find the minimum cost tour.

Algorithm

i. Start.
ii. Read the number of cities and the cost matrix.
iii. Fix city 0 as the starting city.
iv. Generate all possible orders of visiting the remaining cities.
v. Calculate the total cost of each tour, including the return to the starting city.
vi. Find the tour with the minimum cost.
vii. Display the minimum cost and optimal tour.
viii. Stop.

Code

```
#include <iostream>

#include <algorithm>

#include <climits>

using namespace std;

int main() {

   int n;

   cout << "Enter number of cities: ";

   cin >> n;

   int cost[10][10];
```

```
cout << "Enter cost matrix:\n";
for (int i = 0; i < n; i++) {
   for (int j = 0; j < n; j++) {
      cin >> cost[i][j];
   }
}

int path[10], bestPath[10];
for (int i = 0; i < n - 1; i++)
   path[i] = i + 1;
int minCost = INT_MAX;
do {
   int currentCost = cost[0][path[0]];
   for (int i = 0; i < n - 2; i++)
      currentCost += cost[path[i]][path[i + 1]];
   currentCost += cost[path[n - 2]][0];
   if (currentCost < minCost) {
      minCost = currentCost;
      bestPath[0] = 0;
      for (int i = 0; i < n - 1; i++)
         bestPath[i + 1] = path[i];
   }
```

```
    } while (next_permutation(path, path + n - 1));

    cout << "Minimum cost: " << minCost << endl;

    cout << "Optimal tour: ";

    for (int i = 0; i < n; i++)

        cout << bestPath[i] << " -> ";

    cout << "0" << endl;

    return 0;

}
```

Output:

```
Output

Enter number of cities: 4
Enter cost matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Minimum cost: 80
Optimal tour: 0 -> 1 -> 3 -> 2 -> 0
```

Time Complexity:

Time Complexity: O((V-1)!)
Space Complexity: O(V^2)
Here, V is the number of cities.

Conclusion:

The Travelling Salesman Problem was implemented successfully in C++ using the brute-force method to find the minimum-cost tour that visits every city once and returns to the starting city.
Note: Enter city numbers from 0 to V-1. This program assumes all cities are connected and the number of cities is between 2 and 10.